C++ Programming Problem Set

Module: Inline Functions (Optimization, Code Expansion & Compiler Rules)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Inline Functions & Compile-Time Expansion

Integrates: I/O, Operators, Reference Variables, Loops, Arrays & Strings

Instructions & Pedagogical Guidelines

This problem set focuses on C++ inline function mechanics and performance trade-offs. You will write code that demonstrates in-place code replacement, benchmark stack frame call overhead against inline expansion, evaluate compiler rejection rules (loops, recursion, static variables), compare preprocessor macros against type-safe inline templates, and apply inline utilities across array and string processing tasks.

Level 1: Basic Inline Syntax, Arithmetic Utilities & Preprocessor Comparison

Problem 1: Inline Math Utilities & Expansion Inspector

Inline Keyword | Math Expressions | I/O

Create three concise inline functions: `inline int square(int x)`, `inline int cube(int x)`, and `inline double clamp(double val, double minVal, double maxVal)`. In `main()`, prompt the user for numerical inputs in a loop that terminates when the user enters 0. Demonstrate that calling these inline utilities produces zero runtime function call stack frame jump overhead while keeping arithmetic logic cleanly modularized.

inline Keywords | Zero Jump Overhead | Sentinel Loops

Sample Input	Sample Calculation Output
`x = 4, val = 12.5 (min=0, max=10)`	`Square: 16 | Cube: 64 | Clamped Value: 10.00`

Problem 2: Macro Pitfall vs Type-Safe Inline Function

Preprocessor #define | Inline Safety | Side Effects

Compare C-style macro definitions against modern C++ inline functions. Define a preprocessor macro `#define SQUARE_MACRO(x) ((x) * (x))` and an inline function `inline int squareInline(int x)`. Test both in `main()` using arguments with side effects (e.g., a++). Show how the macro expands into `((a++) * (a++))` causing double increments and undefined behavior, whereas the inline function evaluates parameter expressions safely.

#define Macro Pitfalls | Type Safety | Increment Side Effects

Test Case (a = 3)	Behavior & Result Output
`SQUARE_MACRO(a++)`	`Returns: 12 (Evaluated as (3 * 4)) | Final a = 5 (Double Increment Bug!)`
`squareInline(a++)`	`Returns: 9 (Evaluated as (3 * 3)) | Final a = 4 (Safe Execution!)`

Problem 3: Pass-by-Reference Inline Value Swapper

Inline Functions | Reference Variables | In-Place Mutation

Design an inline function `inline void fastSwap(int &a, int &b)` using pass-by-reference aliases. In `main()`, take two integer inputs, call `fastSwap(x, y)` inside a processing loop, and verify that values mutate directly in physical memory without creating unnecessary pointer stack allocation overhead. Print memory addresses of x and y before and after swapping.

Pass-by-Reference Alias | Inline Code Expansion | Direct RAM Mutation

Sample Input	Sample Output
`x = 100, y = 200`	`Before Swap: x=100, y=200`

Level 2: Compiler Rejection Rules, Arrays & String Manipulations

Problem 4: Compiler Heuristic Tester (When Inline is Ignored)

Compiler Cost Heuristic | Loops & Static Vars | Control Flow

The `inline` keyword is a request, not a binding command. Write a program containing four functions marked with `inline`:
1. `inline int add(int a, int b)` — Clean 1-liner (Accepted).
2. `inline void printLoop(int n)` — Contains a `for` loop (Rejected by compiler heuristic).
3. `inline int getID()` — Contains a `static int counter` (Rejected due to static duration).
4. `inline int recurseFactorial(int n)` — Recursive function (Rejected due to unbounded depth).
Document why modern compilers generate standard runtime CALL instructions for items 2–4.

```
// Compiler Heuristic Evaluation: inline int simpleAdd(int a, int b) { return a + b; } // OK: Inlined! inline int loopFunc(int n) { for(int i=0;i<n;i++); } // REJECTED: Contains loop! inline int staticFunc() { static int x=0; return ++x; } // REJECTED: Static storage!
```

Function Category	Compiler Decision Reason
`1-Line Arithmetic`	`INLINED: Zero jump cost; binary footprint tiny.`
`Loops / Static / Recursion`	`REJECTED: Compiled as standard call to prevent binary code bloat.`

Problem 5: In-Place Array Element Transformer

Inline Functions | 1D Array Processing | Const Reference

Implement an inline predicate function `inline bool isEven(int n)` and an inline transformation function `inline void applyTransform(int &val)` that doubles an even integer or triples an odd integer. In `main()`, iterate through a user-supplied array of size N using a range-based `for` loop, applying these inline functions. Print the updated array state.

Inline Predicates | Range-Based For Loop | In-Place Array Update

Input Array	Transformed Output
`[3, 8, 11, 14, 5]`	`Transformed Array: [9, 16, 33, 28, 15]`

Problem 6: String Sanitizer & Vowel Stripper

Inline Functions | std::string | Pass-by-Const-Ref

Write two helper inline functions:
1. `inline bool isVowel(char c)` — Evaluates if a character is a vowel (case-insensitive).
2. `inline char sanitizeChar(char c)` — Replaces non-alphanumeric characters with an underscore.
In `main()`, prompt for a sentence using `std::getline(cin, str)`. Process the string character-by-character using these inline helpers to strip vowels and replace punctuation.

Inline Helper Functions | std::getline Sanitization | Zero Call Overhead

Sample Input Sentence	Sanitized String Output
`"Hello, World! 2026"`	`Processed: "H_ll__ W_rld_ 2026"`

Level 3: Performance Benchmarking, Class Methods & Capstone

Problem 7: Execution Speed Benchmark (Standard vs Inline)

Performance Benchmarking | Stack Latency | Execution Timing

Demonstrate the execution speed advantage of inline expansion over standard function call overhead. Write two identical functions that perform a simple arithmetic operation (e.g., `(a * b) + (a / b)`): one declared standard `int computeStandard(int a, int b)` and one marked `inline int computeInline(int a, int b)`. Invoke both inside a loop executing 10,000,000 times, measuring execution time using high-resolution timers.

```
// High-Resolution Benchmark Loop: auto start = std::chrono::high_resolution_clock::now(); for(long i = 0; i < 10000000; ++i) { result += computeInline(10, 20); } auto end = std::chrono::high_resolution_clock::now(); // Observe elimination of 10,000,000 stack frame setup/teardown cycles!
```

Execution Strategy	Stack Frame Overhead Analysis
`Standard Function Call`	`High Latency: 10,000,000 register pushes, address jumps & RET stack pops.`
`Inline Function Call`	`Zero Latency: Substituted in-place directly into CPU instruction stream.`

Problem 8: Implicit Inlining in Class/Struct Member Methods

Classes & Structs | Implicit Inlining | Encapsulation

In C++, any function defined *inside* a `struct` or `class` body is implicitly treated as `inline` by the compiler without requiring explicit keywords. Define a `Rectangle` structure with `width` and `height`. Implement member functions `double getArea()` and `double getPerimeter()` inside the struct body. Write an external function `inline bool isSquare(const Rectangle &r)` and verify that both implicit and explicit inline calls execute in-place.

Implicit Class Inlining | Getter/Setter Inlining | Struct Reference Parameters

Struct Properties	Output Summary
`Rectangle: width = 5.0, height = 5.0`	`Area: 25.0 | Perimeter: 20.0` `isSquare Check: TRUE (Executed via implicit inlining)`

Problem 9: Cumulative Capstone: Fast Financial Transaction Processor

Comprehensive Integration | Inline Engine | Batch Processing

Build a high-performance financial transaction processor. Define a structure `Transaction` with `id`, `amount`, and `type` (CHAR: 'D' for deposit, 'W' for withdrawal). Design inline helper functions:
1. `inline bool validateTransaction(const Transaction &t)` — Ensures positive amounts.
2. `inline void processTransaction(const Transaction &t, double &balance, double &totalVolume)` — Updates balance directly via reference.
Process a batch array of 5 transactions in `main()`, computing updated account balances and total transaction volume with zero function call overhead.

Inline Batch Processing | Const Reference Validation | Reference Accumulator | Complete Module Capstone

Batch Input Transactions	Processed Account Ledger Output
`Initial Balance: $1,000.00` `T1: +$500 ('D'), T2: -$200 ('W')`	`Final Balance: $1,300.00 | Total Volume Processed: $700.00` `Validation: 100% Valid (Executed via zero-overhead inline engine)`